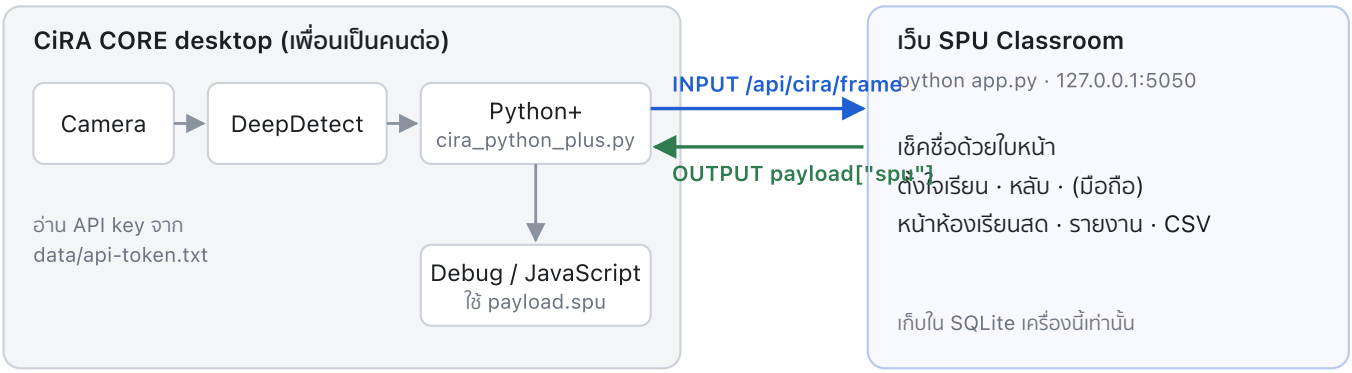
SPU Classroom · คู่มือสำหรับผู้ต่อ flow ฝั่ง CiRA CORE

เชื่อมต่อ CiRA CORE กับเว็บเช็คชื่อและตรวจพฤติกรรม

ส่งผล DeepDetect จาก CiRA เข้าเว็บ (INPUT) และรับผลสรุปกลับเข้า flow ของ CiRA (OUTPUT) ตั้งแต่ติดตั้งจนตรวจรับบนเครื่องจริง



Repository	github.com/Porlager/spu-cira-attendance
เวอร์ชันเอกสาร	19 ตุลาคม 2026 · ใช้กับ branch `main`
สถานะ	API และโค้ด Python+ ทดสอบแล้วด้วยข้อมูลสังเคราะห์และตัวจำลอง ยังไม่ได้ตรวจรับบน CiRA CORE จริง

สรุปสั้นสำหรับคนรีบ

1. **ติดตั้งเว็บบนเครื่องเดียวกับ CiRA CORE (รุ่น desktop)** แล้วรัน `python app.py` เปิด `http://127.0.0.1:5050`
2. **ลงทะเบียนใบหน้า** ผู้ร่วมทดลองที่ยินยอม แล้ว **เปิดคาบ** ในหน้า "ห้องเรียน" (ไม่เปิดคาบ = ข้อมูลถูกปฏิเสธ)
3. **ดู Debug ของ DeepDetect** จดชื่อคลาสที่โมเดลส่งออกมา
4. **หน้า "ตั้งค่า CiRA"** จับคู่ชื่อคลาสกับพฤติกรรม (รูปแบบกรอบตั้งตามเอกสาร CiRA ไว้แล้ว)
5. **วาง Python+ node ต่อจาก DeepDetect** ใส่โค้ดจาก `integrations/cira_python_plus.py` แก้บรรทัด `REPO` บรรทัดเดียว
6. **ตรวจผล** หน้า "ห้องเรียน" ต้องมีกรอบทับคนพอดีและแถบบนเป็นจุดเขียว ส่วน Debug ใน CiRA ต้องเห็น `payload.spu.status = "ok"`

INPUT กับ OUTPUT คืออะไรในงานนี้

ทิศทาง	ข้อมูล	ช่องทาง
INPUT CiRA → เว็บ	ผล DeepDetect ต่อเฟรม (ชื่อคลาส, confidence, กรอบ) + ภาพเฟรมเดียวกัน (ไม่บังคับ)	Python+ → `POST /api/cira/frame`
INPUT CiRA → เว็บ	ผลของ function อื่น เช่น DeepClassif ไว้ดูโครงสร้าง (ไม่นำไปคำนวณ)	`POST /api/cira/output`
OUTPUT เว็บ → flow	ผลสรุปเฟรมล่าสุดใส่กลับใน payload: จำนวนคน, % ตั้งใจ, หลับ, ใครถูกจำได้และทำอะไรอยู่	`payload["spu"]` ให้ node ถัดไปใช้
OUTPUT เว็บ → CiRA	สรุปทั้งห้องช่วง 10 วินาที + รายชื่อพร้อมสถานะ (ใช้ทำแจ้งเตือนได้)	`GET /api/live`
OUTPUT เว็บ → คน	หน้าห้องเรียนสด, รายงานรายนาที/รายคน, ไฟล์ CSV	หน้าเว็บ / `GET /api/sessions/<id>/export`

> **ต้องรันบนเครื่องเดียวกัน** เว็บรับเฉพาะ `127.0.0.1` / `localhost` ถ้า CiRA อยู่อีกเครื่อง ให้ย้ายเว็บไปไว้เครื่อง CiRA อย่าเปลี่ยนเป็น `0.0.0.0` เพราะยังไม่มีระบบล็อกอินและ HTTPS
>
> **ต้องใช้ CiRA CORE รุ่น desktop** รุ่น browser (ciracore.cira-ai.com) ไม่มี Python+ และ REST node

สารบัญ

1. ติดตั้งและเตรียมเว็บ
2. อ่าน Debug ของ DeepDetect
3. ตั้งค่าให้ตรงกับโมเดล
4. INPUT + OUTPUT ด้วย Python+ node
5. สัญญา HTTP (ไม่ใช้ Python+)
6. OUTPUT: อ่านผลกลับ
7. ข้อผิดพลาดที่พบบ่อย
8. ขั้นตอนทดสอบและตรวจรับ
9. ข้อห้ามด้านความปลอดภัย
10. ภาคผนวก: endpoint และแหล่งอ้างอิง

1. ติดตั้งและเตรียมเว็บ

ต้องใช้ Python 3.9–3.12 บนเครื่องที่รัน CiRA CORE (เว็บใช้ Python ของตัวเอง แยกจาก Python ที่ CiRA ใช้)

```
git clone https://github.com/Porlager/spu-cira-attendance.git
cd spu-cira-attendance
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python app.py                      # เปิด http://127.0.0.1:5050
```

- ครั้งแรกระบบสร้างโฟลเดอร์ `data/` พร้อม `data/api-token.txt` (API key) และฐานข้อมูล `data/attendance.sqlite3`
- เปลี่ยน port ด้วย `PORT=5051 python app.py` เก็บข้อมูลที่อื่นด้วย `SPU_DATA_DIR=/path python app.py`
- path เต็มของไฟล์ API key ดูได้ในหน้า "ตั้งค่า CiRA" หัวข้อ "การเชื่อมต่อ"

เตรียมก่อนส่งข้อมูล

1. หน้า **ลงทะเบียนใบหน้า**: ถ่าย 3–8 ภาพต่อคน ติ๊กยินยอม แล้วบันทึก (เฉพาะคนที่ยินยอม)
2. หน้า **ห้องเรียน**: ใส่ชื่อคาบ เช่น `CPE447 · ห้อง 11-501` แล้วกด "เปิดคาบเรียน"
3. เปิดได้ครั้งละหนึ่งคาบ ปิดด้วยปุ่ม "ปิดคาบ" มุมขวาบน หลังปิดแล้ว CiRA ส่งเข้าคาบเดิมไม่ได้

ทดลองก่อนมีโมเดล (ไม่บังคับ)

ใช้ตัวจำลองดูว่าหน้าเว็บทำงานอย่างไร ระบบเปิด "คาบจำลอง" แยกให้และติดป้าย **ข้อมูลจำลอง** ทุกจุด ตัวจำลองส่งกรอบรูปแบบเดียวกับ DeepDetect จริง

```
python integrations/simulate_cira.py --seconds 120 --with-identities
```

ตัวจำลองไม่ทำงานถ้ามีคาบจริงเปิดอยู่ และเว็บไม่รับข้อมูลจำลองเข้าคาบจริง ตัวเลขจากตัวจำลองเป็นค่าสุ่ม ห้ามใช้เป็นผลทดลอง

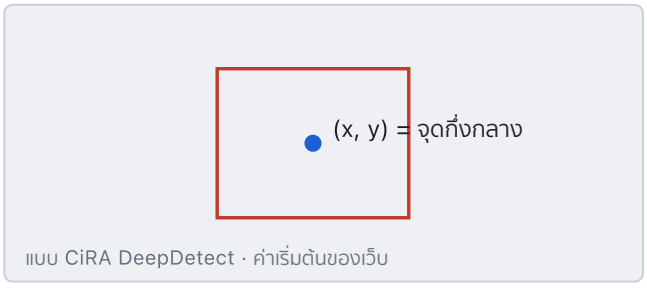
2. อ่าน Debug ของ DeepDetect

ต่อ Debug node หลัง DeepDetect แล้วดูโครงสร้างจริงของรุ่นที่ติดตั้ง ตามเอกสาร CiRA CORE 2025 (JavaScript Update หน้า 27–28) ผล DeepDetect หน้าตาประมาณนี้:

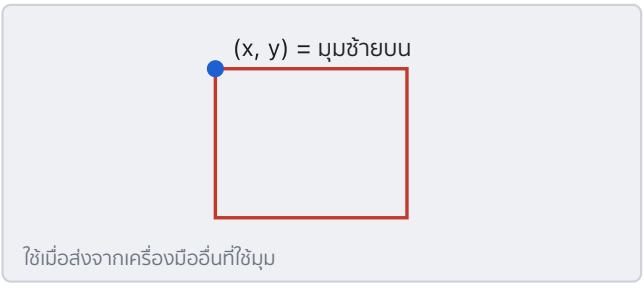
```
{
  "DeepDetect": {
    "count": 2,
    "counts": {"awake": 1, "sleep": 1},
    "im_w": 1280, "im_h": 720,
    "objects": [
      {"id": 0, "name": "awake", "confidence": 0.91, "x": 290, "y": 390, "w": 180, "h": 260},
      {"id": 1, "name": "sleep", "confidence": 0.84, "x": 650, "y": 390, "w": 180, "h": 260}
    ]
  }
}
```

ตัวอย่างนี้เขียนขึ้นตามรูปแบบในเอกสาร ตัวเลขไม่ใช่ผลจากโมเดลจริง

field	ตามเอกสาร CiRA	สิ่งที่ต้องเช็คบนเครื่องจริง
`name`	ชื่อคลาสที่ตั้งตอนเทรนโมเดล	จดทุกชื่อ ไปจับคู่ในหน้าตั้งค่า
`x` , `y`	**จุดกึ่งกลางกรอบ** หน่วยพิกเซล	กรอบบนเว็บต้องทับคนพอดี
`w` , `h`	กว้าง/สูงของกรอบ พิกเซล	—
`confidence`	0–1 (บนจอแสดงเป็น % แต่ค่าใน payload เป็น 0–1)	ถ้าเห็นค่าเกิน 1 ให้ตั้งสเกล 0–100
`im_w` , `im_h`	ขนาดภาพที่ใช้ตรวจ เว็บใช้แปลงพิกัดให้อัตโนมัติ	ถ้าไม่มี ต้องแนบภาพหรือส่ง width/height
key `DeepDetect`	บาง build ใช้ชื่ออื่น เช่น `DeepDetectCPU`	ถ้ามี detector เดียว ระบบหาเอง ถ้ามีหลายตัวต้องระบุ



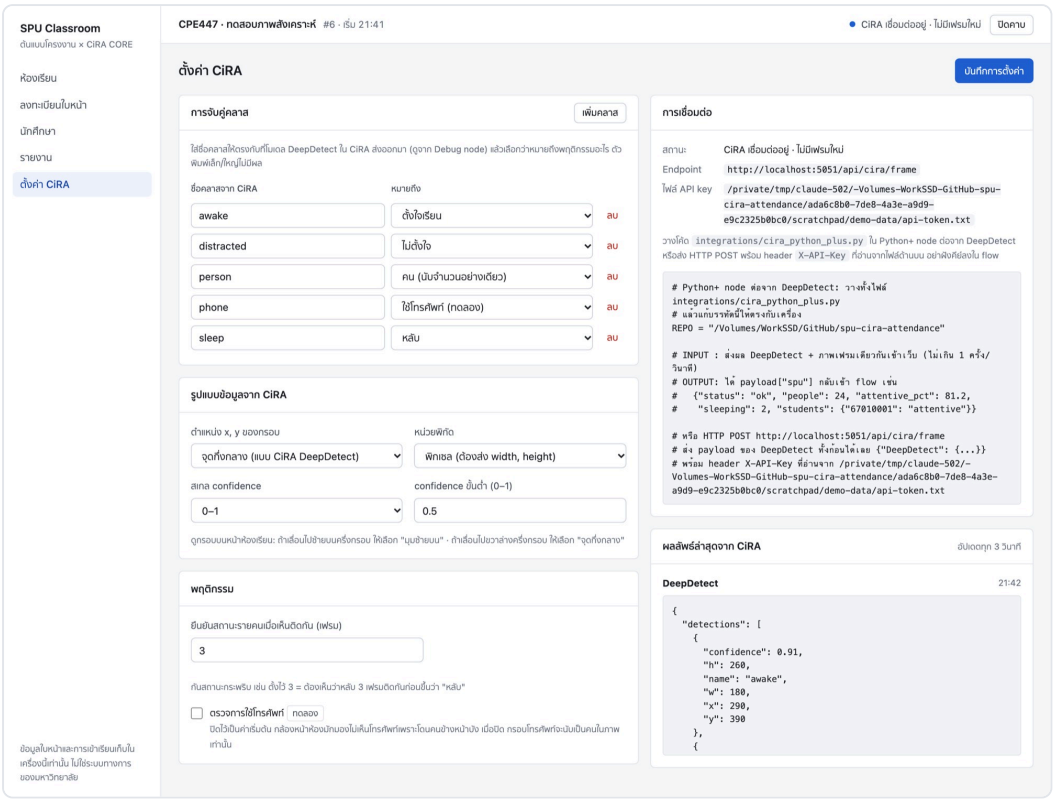
แบบ CiRA DeepDetect · ค่าเริ่มต้นของเว็บ



ใช้เมื่อส่งจากเครื่องมืออื่นที่ใช้มุม

3. ตั้งค่าให้ตรงกับโมเดล

เปิดหน้า **ตั้งค่า CiRA** แก้แล้วกด "บันทึกการตั้งค่า" มีผลกับเฟรมถัดไปทันที ไม่ต้องรีสตาร์ท



หน้าตั้งค่า CiRA (ภาพจากเครื่องทดสอบ path ในเครื่องจริงจะต่างไป): ซ้ายคือการจับคู่คลาสและรูปแบบข้อมูล ขวาคือ endpoint, path ของ API key และผลล่าสุดที่ CiRA ส่งมา

การจับคู่คลาส

ใส่ชื่อคลาสให้ตรงกับ Debug (ตัวพิมพ์เล็ก/ใหญ่ไม่มีผล) แล้วเลือกว่าหมายถึงอะไร ชื่อที่ใส่ไว้ตอนนี้ (`awake` , `distracted` , `sleep` , `phone` , `person`) เป็นตัวอย่าง:

เลือก	ความหมาย
ตั้งใจเรียน	กรอบพฤติกรรมของคนหนึ่งคน นับเป็นตัวตั้งของ % ตั้งใจเรียน
ไม่ตั้งใจ	เช่น หันคุย มองที่อื่น
หลับ	นับในยอดหลับของทั้งห้อง และผูกกับชื่อถ้าจำหน้าได้
ใช้โทรศัพท์ (ทดลอง)	นับเฉพาะเมื่อเปิดสวิตช์ "ตรวจการใช้โทรศัพท์" ถ้าปิด กรอบนี้นับเป็นคนในภาพเท่านั้น
คน (นับจำนวนอย่างเดียว)	กรอบ person ทั่วไป ใช้นับจำนวนคน ไม่นับใน % ตั้งใจ
ไม่นับ	ข้ามคลาสนี้ เช่น `chair` , `book`

คลาสที่ CiRA ส่งมาแต่ยังไม่ได้จับคู่ จะขึ้นเป็นปุ่ม "+ ชื่อคลาส" ใต้ตาราง กดเพื่อเพิ่มได้ทันที ระหว่างนั้นระบบข้ามคลาสนั้นไปก่อน

สูตร: ตั้งใจเรียน (%) = ตั้งใจ ÷ (ตั้งใจ + ไม่ตั้งใจ + หลับ + ใช้โทรศัพท์ถ้าเปิด) · จำนวนคน = ค่ามากกว่าระหว่างจำนวนกรอบ "คน" กับจำนวนกรอบพฤติกรรม จึงใช้โมเดล person กับโมเดลพฤติกรรมพร้อมกันได้โดยไม่นับซ้ำ

รูปแบบข้อมูลและพฤติกรรม

ช่อง	ค่าเริ่มต้น	หมายเหตุ
ตำแหน่ง x, y ของกรอบ	จุดกึ่งกลาง	ตรงกับ DeepDetect ถ้ากรอบเลื่อนไปซ้ายบนครึ่งกรอบ แปลว่าข้อมูลเป็นมุม ให้เลือก "มุมซ้ายบน"
หน่วยพิกัด	พิกเซล	ใช้ `im_w` / `im_h` ใน payload หรือภาพที่แนบมาแปลงให้
สเกล confidence	0–1	หรือ 0–100 ระบบหารให้เอง ห้ามหารซ้ำใน Python+
confidence ขั้นต่ำ	0.5	ต่ำกว่านี้ไม่นับ
ยืนยันสถานะรายคน (เฟรม)	3	ต้องเห็นพฤติกรรมเดิมติดกัน N เฟรมก่อนเปลี่ยนสถานะ กันกระพริบ
ตรวจการใช้โทรศัพท์	ปิด	กล้องหน้าห้องมักโดนบัง ทดลองแล้วค่อยตัดสินใจ

INPUT OUTPUT 4. Python+ node (แนะนำ)

Python+ node ของ CiRA ใช้ฟังก์ชัน `handle_request(req)` ที่อ่านข้อมูลด้วย `getReqData(req)` และส่งต่อด้วย `makeRespData(payload, img)` (Intro 2025 หน้า 14) ไฟล์ `integrations/cira_python_plus.py` ใน repo เขียนตามแบบนั้นแล้ว:

```
import sys

REPO = "/absolute/path/spu-cira-attendance"   # <-- แก้บรรทัดนี้ให้ตรงกับเครื่อง
BASE_URL = "http://127.0.0.1:5050"
SEND_IMAGE = True        # False = ส่งเฉพาะกรอบ (เช็คชื่อรายคนไม่ได้)
TIMEOUT = 2              # วินาที กัน flow ค้างถ้าเว็บไม่ตอบ

sys.path.append(REPO + "/integrations")
from cira_bridge import error_message, flow_output, send_detections

TOKEN_FILE = REPO + "/data/api-token.txt"
_last = {"status": "waiting"}

def handle_request(req):
    global _last
    payload, img = getReqData(req)
    frame = img if SEND_IMAGE and getattr(img, "size", 0) else None
    try:
        result = send_detections(payload, TOKEN_FILE, frame_bgr=frame,      # INPUT
                                 base_url=BASE_URL, timeout=TIMEOUT)
        if result is not None:
            _last = dict(flow_output(result), status="ok")
    except Exception as error:      # เว็บปิด/ยังไม่เปิดคาบ: flow ต้องเดินต่อ
        _last = {"status": "error", "error": error_message(error)}
    payload["spu"] = _last                                                  # OUTPUT
    return makeRespData(payload, img)
```

สิ่งที่โค้ดนี้ทำ

- **INPUT:** ส่ง payload ของ DeepDetect ทั้งก้อน + ภาพ `img` เข้าเว็บ ระบบหา `objects` , `im_w` , `im_h` เอง ย่อภาพเหลือกว้างไม่เกิน 1280 px
- **ส่งจริงไม่เกิน 1 ครั้งต่อวินาที** เฟรมที่เว้นไว้จะได้ผลล่าสุดซ้ำ จึงไม่ถ่วง flow ทุกเฟรม
- **OUTPUT:** ใส่ผลกลับใน `payload["spu"]` ให้ node ถัดไปใช้ (รายละเอียดหัวข้อ 6.1)
- **ไม่ทำให้ flow หยุด** ถ้าเว็บปิดหรือส่งไม่ผ่าน จะได้ `payload["spu"]["status"] = "error"` พร้อมเหตุผล

> **ต้องเช็คบนเครื่องจริงก่อน (ยังไม่ได้ทดสอบกับ CiRA ตัวจริง):**
>
> - ใส่ `print(type(img), getattr(img, "shape", None))` ชั่วคราว ภาพควรเป็น numpy BGR เช่น `(720, 1280, 3)` ถ้าติ๊ก "Not use image" ไว้ `img` อาจว่าง โค้ดจะส่งเฉพาะกรอบ
> - Python ที่ Python+ ใช้ (เลือกได้ด้วยปุ่ม Browse) ต้องมี `opencv` และ `numpy` ซึ่งปกติมีอยู่แล้ว bridge ใช้แค่ standard library นอกนั้น รองรับ Python 3.8 ขึ้นไป
> - ถ้ามี detector หลายตัวใน payload ให้ส่ง `function="ชื่อ key"` เพิ่มใน `send_detections(...)`

ส่งผลของ function อื่นมาดู

เช่น DeepClassif ส่งมาดูโครงสร้างในหน้าตั้งค่าได้ (ไม่นำไปคำนวณ เก็บในหน่วยความจำ ขนาดไม่เกิน 64 KB):

```
from cira_bridge import send_output
send_output("DeepClassif", payload.get("DeepClassif"), TOKEN_FILE, timeout=2)
```

พารามิเตอร์ของ send_detections

ชื่อ	ค่าเริ่มต้น	ใช้ทำอะไร
`objects`	(ต้องมี)	payload ของ DeepDetect ทั้งก้อน, `{"objects": [...]}` หรือ list ของ objects
`token_file`	(ต้องมี)	path ของ `data/api-token.txt`
`frame_bgr`	`None`	ภาพเฟรมเดียวกัน ใส่แล้วได้: เช็คชื่อด้วยใบหน้า, สรุปรายคน, เห็นภาพจริงบนเว็บ
`width` , `height`	จาก `im_w` / `im_h`	ใส่เองเมื่อ payload ไม่มี `im_w` / `im_h` และไม่แนบภาพ
`function`	`"DeepDetect"`	key ใน payload และชื่อที่แสดงบนเว็บ
`min_interval`	`1.0`	ส่งจริงไม่ถี่กว่านี้ (วินาที)
`timeout`	`10`	ใน Python+ ใช้ 2 วินาทีพอ
`base_url`	`http://127.0.0.1:5050`	เปลี่ยนเมื่อใช้ port อื่น

กติกาภาพที่แนบ

- ต้องเป็น **เฟรมเดียวกับผลตรวจ** ย่อได้ แต่ **ห้าม crop** ไม่อย่างนั้นกรอบจะไม่ตรงหน้า
- ภาพเก็บในหน่วยความจำเฉพาะเฟรมล่าสุด ไม่เขียนลงดิสก์ ปิดคาบแล้วหายทันที
- ระบบผูกใบหน้ากับกรอบพฤติกรรมที่ **จุดกึ่งกลางใบหน้าอยู่ในกรอบ** ถ้าซ้อนหลายกรอบเลือกกรอบเล็กสุด ถ้าสองคนอยู่ในกรอบเดียว ระบบจะไม่ผูกกับใคร
- คนที่ฟุบหลับมักจำหน้าไม่ได้ จะอยู่ในยอดหลับของห้องแต่ไม่มีชื่อ เป็นพฤติกรรมที่ตั้งใจไว้

INPUT 5. สัญญา HTTP (กรณีไม่ใช้ Python+)

ใช้เมื่อส่งจากเครื่องมืออื่น ต้องมี header `X-API-Key` ที่อ่านจากไฟล์ทุกครั้ง **ห้ามพิมพ์คีย์ลงใน flow** รับทั้ง `POST` และ `PUT` และรับได้สองรูปแบบ:

แบบที่ 1: payload ของ CiRA ทั้งก้อน

```
POST http://127.0.0.1:5050/api/cira/frame
Content-Type: application/json
X-API-Key: <อ่านจาก data/api-token.txt>

{"DeepDetect": {"im_w": 1280, "im_h": 720,
                "objects": [{"name": "sleep", "confidence": 0.82, "x": 140, "y": 165, "w": 80,
"h": 90}]},
 "image": "<base64 JPEG/PNG ไม่บังคับ>"}
```

ห่อเป็น `{"payload": {...}}` ก็ได้ ถ้ามี detector หลาย key ให้เพิ่ม `"function": "ชื่อ key"`

แบบที่ 2: รายการกรอบที่จัดเอง

```
{"function": "DeepDetect", "width": 1280, "height": 720,
 "detections": [{"name": "sleep", "confidence": 0.82, "x": 140, "y": 165, "w": 80, "h": 90}]}
```

field	ชนิด	กติกา
`objects` / `detections`	list	ไม่เกิน 300 รายการ ส่ง `[]` ได้เมื่อไม่พบใคร
`└ name`	string	1–64 ตัวอักษร
`└ confidence`	number	0 ถึงสเกลที่ตั้งไว้ (1 หรือ 100)
`└ x, y, w, h`	number	w, h > 0 กรอบที่ล้นภาพจะถูกตัดให้อยู่ในภาพ
`width` , `height`	integer	1–10000 ใช้ `im_w` / `im_h` แทนได้ ต้องมีอย่างใดอย่างหนึ่งเมื่อหน่วยเป็นพิกเซล
`image`	string	base64 หรือ data URL ไม่เกิน 3 MB ด้านยาวไม่เกิน 4096 px
`session_id`	integer	ไม่บังคับ ถ้าใส่ต้องเป็นคาบที่เปิดอยู่

> **REST node ของ CiRA (RestPutJson):** มีใน palette รุ่น desktop แต่ไม่พบเอกสารวิธีตั้ง URL และ header ถ้าใส่ header `X-API-Key` ไม่ได้ จะใช้ส่งเข้าเว็บนี้ไม่ได้ ให้ใช้ Python+ แทน · JavaScript node ไม่มีเอกสารเรื่องการเรียก HTTP จึงไม่แนะนำ

ทางเดิม (ยังใช้ได้)

- `POST /api/cira/observations` `{"session_id": 1, "people_count": 12}` บันทึกจำนวนคนอย่างเดียว
- `POST /api/recognize` `{"session_id": 1, "image": "..."}` เช็คชื่อจากภาพอย่างเดียว ไม่ตรวจพฤติกรรม

OUTPUT 6. อ่านผลกลับจากเว็บ

6.1 ใน flow ของ CiRA: payload["spu"]

หลัง Python+ node ทุกเฟรมจะมี `payload.spu` ต่อ Debug เพื่อดู หรือใช้ใน node ถัดไป:

```
"spu": {
  "status": "ok", "seq": 41,
  "people": 24, "attentive_pct": 79.2,
  "sleeping": 3, "inattentive": 2, "phone": 0,
  "students": {"67010001": "attentive", "67010007": "sleeping"},
  "ignored_labels": []
}
```

field	ความหมาย
`status`	`ok` · `waiting` (ยังไม่ได้ส่งครั้งแรก) · `error` (ดู `error`)
`people`	จำนวนคนในเฟรมที่ส่งล่าสุด
`attentive_pct`	% ตั้งใจของเฟรมนั้น `null` ถ้าไม่มีกรอบพฤติกรรม
`sleeping` / `inattentive` / `phone`	จำนวนกรอบแต่ละพฤติกรรม (`phone` เป็น 0 เมื่อปิดฟีเจอร์)
`students`	รหัสนักศึกษาที่จำหน้าได้ในเฟรมนี้ → พฤติกรรมที่ผูกได้ (`null` = จำได้แต่ไม่อยู่ในกรอบใด)
`ignored_labels`	คลาสที่ยังไม่ได้จับคู่ ไปเพิ่มในหน้าตั้งค่า

ตัวอย่างใช้ใน JavaScript node ถัดไป (ใช้ `payload` และ `print` ตามเอกสาร JavaScript Update 2025):

```
if (payload.spu && payload.spu.status == "ok" && payload.spu.sleeping >= 3) {
  print("มีคนหลับ " + payload.spu.sleeping + " คน");
}
```

ตัวเลขในตัวอย่างเป็นข้อมูลสมมติ ค่าในหนึ่งเฟรมแกว่งได้ ถ้าจะทำแจ้งเตือนจริงใช้ค่าเฉลี่ยจากข้อ 6.2

6.2 สรุปทั้งห้องแบบนิ่ง: GET /api/live

```
from cira_bridge import get
live = get("/api/live", TOKEN_FILE, timeout=2)   # เรียกไม่เกิน 1 ครั้ง/วินาที
k = live["kpi"]
alert = (k["sleeping"] or 0) >= 3                 # None เมื่อยังไม่มีข้อมูล
present = [r["id"] for r in live["roster"] if r["present"]]
```

field	ความหมาย
`session`	คาบที่เปิดอยู่ (`id`, `title`, `started_at`, `simulated`) หรือ `null`

field	ความหมาย
`cira.status`	`live` (มีเฟรมใน 5 วิ) · `error` (ถูกปฏิเสธล่าสุด ดู `last_error`) · `connected` · `offline`
`kpi.registered / present`	จำนวนที่ลงทะเบียน / เช็คชื่อแล้วในคาบนี้
`kpi.attentive_pct`	% ตั้งใจ รวมทุกเฟรมใน 10 วินาทีล่าสุด
`kpi.sleeping / inattentive / people`	ค่าเฉลี่ยต่อเฟรมใน 10 วินาที (`phone` เป็น `null` เมื่อปิด)
`roster[]`	`id`, `name`, `present`, `seen_at`, `status`, `status_seconds`, `frames`, `shares`
`roster[].status`	`attentive` · `inattentive` · `sleeping` · `phone` · `unknown` (ยังไม่ยืนยัน) · `not_seen` (ไม่เห็นเกิน 20 วิ)

6.3 Endpoint อื่นสำหรับ OUTPUT

`GET /api/status`	คาบที่เปิด + สถานะการเชื่อมต่อ CiRA (เบา ใช้ตรวจว่าต่อติดไหม)
`GET /api/sessions`	รายการคาบทั้งหมด พร้อมจำนวนเฟรมและจำนวนคนที่มา
`GET /api/sessions/<id>/report`	สรุปคาบ ไทม์ไลน์รายนาที สัดส่วนพฤติกรรมรายคน
`GET /api/sessions/<id>/export`	ไฟล์ CSV (UTF-8 เปิดใน Excel ได้)

ทุก endpoint ต้องมี header `X-API-Key` และเรียกได้จากเครื่องเดียวกันเท่านั้น

7. ข้อผิดพลาดที่พบบ่อย

เว็บตอบ HTTP 400 พร้อม `{"error": "..."}` ภาษาไทย ข้อความเดียวกันจะอยู่ใน `payload.spu.error` และแถบบนของหน้าเว็บจะขึ้นจุดแดง

ข้อความ	แปลว่า / วิธีแก้
`Connection refused` / `timed out`	เว็บไม่ได้รันอยู่ หรือ port ไม่ตรง รัน `python app.py` แล้วตรวจ `BASE_URL`
`HTTP 401: API key required`	คีย์ไม่ตรง ตรวจ `REPO` ว่าชี้ไปที่ repo ที่รันเว็บอยู่ (ถ้าลบโฟลเดอร์ data คีย์จะเปลี่ยน)
`HTTP 403: Localhost only`	เรียกผ่าน IP เครื่อง ใช้ `127.0.0.1` หรือ `localhost` เท่านั้น
`ยังไม่มีคาบเรียนที่เปิดอยู่ เปิดคาบในหน้าเว็บก่อน`	เปิดคาบในหน้าห้องเรียน
`คาบเรียนไม่ตรงกับคาบที่กำลังเปิดอยู่`	ส่ง `session_id` เก่า เอาออก
`ไม่พบผลตรวจจับ ต้องมี detections หรือ payload ที่มี <ชื่อ node>.objects ...`	payload ไม่มี objects หรือมี detector หลายตัว ระบุ `function`
`ต้องส่ง width และ height (พิกเซล) ของภาพที่ใช้ตรวจจับ`	payload ไม่มี `im_w` / `im_h` แนบภาพหรือส่ง width/height
`confidence ต้องอยู่ระหว่าง 0–1 ตรวจการตั้งค่า confidence scale`	CiRA ส่ง 0–100 ไปตั้ง "สเกล confidence" เป็น 0–100
`confidence, x, y, w, h ต้องเป็นตัวเลข`	บางรุ่นส่งเป็นข้อความ แปลงเป็น float ก่อนส่ง
`w และ h ต้องมากกว่า 0`	กรอบขนาดศูนย์ กรองทิ้งก่อนส่ง
`คาบนี้เป็นคาบจำลอง รับเฉพาะข้อมูลจำลอง`	ตัวจำลองเปิดคาบค้างไว้ ปิดคาบนั้นแล้วเปิดคาบจริง
`ภาพไม่ถูกต้องหรือใหญ่เกิน 3 MB`	ภาพใหญ่หรือ encode ผิด ใช้ `frame_bgr=` ของ bridge ซึ่งย่อและ encode ให้
`Expected payload.DeepDetect.objects ...` (จาก bridge)	payload ไม่ใช่รูปแบบที่คาด `print(payload)` ดูโครงสร้างจริง

อาการที่ไม่มี error แต่ผลไม่ถูก

กรอบเลื่อนไปซ้ายบนครึ่งกรอบ	ข้อมูลเป็นแบบมุม เลือก "มุมซ้ายบน"
กรอบเลื่อนไปขวาล่างครึ่งกรอบ	ข้อมูลเป็นจุดกึ่งกลาง เลือก "จุดกึ่งกลาง"
กรอบเล็กกระจุกมุมซ้ายบน	CiRA ส่งสัดส่วน 0–1 แต่ตั้งเป็นพิกเซล

กรอบใหญ่ล้นภาพ	width/height ที่ใช้ไม่ใช่ขนาดของภาพที่ตรวจ (เช่น ใช้ ROI)
ตัวเลขเป็น — ตลอด	ทุกคลาสยังไม่ได้จับคู่ ดูปุ่ม "+ คลาส" ในหน้าตั้งค่า
เข้าเรียน 0 แม้เห็นหน้า	ไม่ได้แนบภาพ หรือหน้าเล็ก/ไกลเกินไป หรือยังไม่ได้ลงทะเบียน
flow ช้าลงทุก 1 วินาที	รอเว็บตอบ ลด `TIMEOUT` หรือตั้ง `SEND_IMAGE = False` ชั่วคราว เพื่อดูว่าภาพเป็นสาเหตุไหม

8. ขั้นตอนทดสอบและตรวจรับ

ทดสอบจาก terminal ก่อนต่อ flow

```
# 1) ต่อถึงเว็บไหม (ดูผลในหน้าตั้งค่า "ผลลัพธ์ล่าสุดจาก CiRA")
python integrations/cira_bridge.py --ping

# 2) ส่ง payload ที่คัดลอกจาก Debug มาเป็นไฟล์ (อย่า commit ไฟล์จากห้องจริง)
python integrations/cira_bridge.py payload.json

# 3) ส่งพร้อมภาพเฟรมเดียวกัน
python integrations/cira_bridge.py payload.json --image frame.jpg
```

เกณฑ์ตรวจรับบน CiRA จริง

- [ ] Debug หลัง Python+ เห็น `payload.spu.status = "ok"`
- [ ] `img` ใน Python+ เป็นภาพ BGR (ดูจาก `img.shape`)
- [ ] ชื่อคลาสจับคู่ครบ ไม่มีปุ่ม "+ คลาส" ค้างในหน้าตั้งค่า
- [ ] กรอบบนหน้าห้องเรียนทับคนในภาพพอดี
- [ ] ภาพ 0 / 1 / หลายคน: จำนวนคนตรงกับที่ตานับ
- [ ] นักศึกษาที่ลงทะเบียนหน้าตรง: ถูกเช็คชื่อ และมีพฤติกรรมในรายชื่อ
- [ ] คนที่ไม่ลงทะเบียน: ไม่ถูกเช็คชื่อเป็นคนอื่น
- [ ] ฟุบหลับ: ยอดหลับของห้องเพิ่ม (ไม่มีชื่อถือว่าถูกตามออกแบบ)
- [ ] ก้มเขียนหนังสือ / หันคุย: จดว่าโมเดลแยกจากหลับได้หรือไม่
- [ ] ปิดเว็บระหว่าง flow รัน: flow ต้องเดินต่อ และ `payload.spu.status = "error"`
- [ ] ส่งก่อนเปิดคาบหรือหลังปิดคาบ: ถูกปฏิเสธ และแถบบนแสดงเหตุผล
- [ ] (ถ้าเปิด) ใช้โทรศัพท์จากมุมกล้องจริง: มองเห็นกี่ครั้งจากกี่ครั้ง เพื่อตัดสินใจว่าจะเก็บฟีเจอร์ไหม

> บันทึกผลจริงลงรายงานโครงงาน ห้ามอ้างว่าเชื่อมสำเร็จจากการทดสอบ API หรือข้อมูลจำลองอย่างเดียว

9. ข้อห้ามด้านความปลอดภัยและข้อมูลส่วนตัว

1. **ห้ามพิมพ์ API key ลงใน flow หรือโค้ด** ให้อ่านจากไฟล์ทุกครั้ง flow ที่อัปโหลดขึ้น GitHub ต้องไม่มีคีย์
2. **ห้ามเปิดเว็บให้เครื่องอื่นเข้าถึง** (ห้ามเปลี่ยน host เป็น `0.0.0.0`) เพราะยังไม่มีระบบล็อกอินและ HTTPS
3. **ห้าม commit** โฟลเดอร์ `data/` , ภาพ, payload หรือ CSV จากห้องเรียนจริง repo นี้เป็น public
4. **ลงทะเบียนเฉพาะคนที่ยินยอม** และลบข้อมูลได้เมื่อขอ (หน้า "นักศึกษา" ปุ่มลบ)
5. **ผลพฤติกรรมเป็นการประเมินของโมเดล** ห้ามใช้ตัดคะแนนหรือลงโทษ แจ้งนักศึกษาก่อนทดลอง

ภาคผนวก: สรุป endpoint

ทุกเส้นทางใต้ `/api/` ต้องมี `X-API-Key` · base URL `http://127.0.0.1:5050`

ทิศ	endpoint	ใช้ทำอะไร
IN	`POST` / `PUT /api/cira/frame`	ผล DeepDetect + ภาพ → เช็คชื่อ, พฤติกรรม, ภาพสด
IN	`POST /api/cira/output`	ผล function อื่นไว้ดูโครงสร้าง (หน่วยความจำ ≤ 64 KB)
IN	`POST /api/cira/observations`	ทางเดิม: จำนวนคนอย่างเดียว
IN	`POST /api/recognize`	ทางเดิม: เช็คชื่อจากภาพอย่างเดียว
OUT	คำตอบของ `/api/cira/frame`	ผลเฟรมนั้น → `payload["spu"]`
OUT	`GET /api/live`	KPI 10 วินาที, รายชื่อพร้อมสถานะ, เฟรมล่าสุด
OUT	`GET /api/live/frame`	ภาพ JPEG เฟรมล่าสุด (404 ถ้าไม่มี)
OUT	`GET /api/status`	คาบที่เปิด + สถานะ CiRA
OUT	`GET /api/sessions/<id>/report` · `/export`	สรุปคาบ ไทม์ไลน์ รายคน · CSV
OUT	`GET /api/cira/outputs`	ผลล่าสุดของแต่ละ function
ตั้งค่า	`GET` / `PUT /api/settings`	การจับคู่คลาสและรูปแบบข้อมูล (ปกติแก้ผ่านหน้าเว็บ)

ไฟล์ที่เกี่ยวข้องใน repo

`integrations/cira_python_plus.py`	โค้ดวางใน Python+ node (INPUT + OUTPUT)
`integrations/cira_bridge.py`	`send_detections` , `flow_output` , `send_output` , `get` และ CLI
`integrations/simulate_cira.py`	ตัวส่งข้อมูลจำลอง
`behavior.py`	กติกาแปลงคลาส normalize กรอบ จับคู่ใบหน้า กันกระพริบ
`docs/WEB_PREVIEW.pdf`	ภาพตัวอย่างทุกหน้าเว็บ

แหล่งอ้างอิง

- CiRA CORE JavaScript Update 2025 หน้า 7–29 (payload, DeepDetect fields): archive.cira-ai.com/other/CiRA_CORE_AYC_2025_Share/
- CiRA CORE Intro AYC 2025 หน้า 9–14 (node palette, Python+ template): โฟลเดอร์เดียวกัน
- CiRA CORE ตอนที่ 24 แนะนำการใช้ Python+ (YouTube) และ github.com/CiRA-AMI/cira-core-python-plus-example

ส่วนที่เอกสารไม่ได้ระบุ (ชนิดของ `img` , การตั้งค่า RestPutJson) ต้องตรวจบนเครื่องจริงตามหัวข้อ 4 และ 8